Selective Routing under Ownership Ambiguity: A Public Benchmark and an Agent Study

Anonymous Author(s)

ABSTRACT

Bug reports are routed by hand: a reporter picks a component, a triager moves it when that choice was wrong, and the component decides which team fixes the bug. We show that the usual way of scoring this task hides the only part of it that is hard. On a public Mozilla Bugzilla benchmark, simply predicting the team that owns the component the reporter already chose reproduces the final team for 83.1% of the bugs, more than any trained model we tried (best: 62.5%). That figure is earned on reports that were never reassigned, where the same baseline scores 100.0%; on reports that were moved away from a concrete component it collapses to 32.8%, below the 55.6% of TF-IDF with logistic regression. Overall accuracy therefore rewards the method that adds least.

We then ask whether a tool-using LLM agent can do better exactly there. On the same bugs the agent reaches 78.9% overall and 71.2% on the reassigned subgroup — 15.5points above the best classical model and 38.4 points above the reporter baseline — with the best risk-coverage curve of any method we evaluate (AURC 0.077, against 0.184 for TF-IDF with logistic regression and 0.333 for the reporter baseline). It is also the one learned method that declines more often on the reassigned bugs than elsewhere, 22.6% against 19.0%, a lift of 1.13, where every classical method declines on them at 0.69–0.90 times its own average rate. That margin is narrower than the accuracy gain, and we report it as such. We release the benchmark, the baselines and the prompts.

1 INTRODUCTION

Bug reports are routed by hand. A reporter picks a component, a triager moves the report when that choice was wrong, and the component decides which team eventually fixes it. A routing decision can therefore fail in two ways: the report was filed under the wrong component, or the right component is hard to identify from the text.

Prior work treats routing as text classification and reports overall accuracy [6, 11, 13, 15, 16]. On our data that framing is misleading. The strongest signal available at filing time is the component the reporter already chose: predicting the team that owns it reproduces the final team for 83.1% of the bugs in our 2026 test period, against 62.5% for TF-IDF with logistic regression and less for every other trained model we tried (Table 5). An accuracy leaderboard would conclude that routing is nearly solved and that no trained model is worth deploying. Production systems take the same view: BugBug in Firefox applies a confidence threshold and assigns only what it is sure of [3].

The aggregate hides the only subgroup where a model helps. Splitting those same bugs by whether their component ever moved shows that the reporter baseline collapses exactly where routing is hard: 100.0% on bugs that were never reassigned, but 32.8% on the bugs that were moved away from a concrete component, where TF-IDF+LR reaches 55.6% (Table 7). The baseline's 83% is earned almost entirely on the easy majority of reports.

The second half of the problem is that the classical methods cannot tell when they are wrong. Ranking bugs by a trained model's confidence and answering only the most confident fraction does raise its accuracy, but every classical method declines on the reassigned subgroup at only 0.69–0.90 times the rate at which it declines overall: it answers the hard bugs *more* readily than the easy ones (Table 7). Its confidence does not track the ambiguity it would need to detect. The reporter baseline is the other exception (lift 1.65), and only because its confidence is a class prior rather than a text-based estimate.

That is the question this paper asks. Can an LLM agent, given tools and asked for candidate teams with evidence, identify ambiguous reports better than a confidence threshold can? Recent systems route incidents with single LLMs [17], with several agents that negotiate [19], and with one agent per team [18]; abstention has been used to cut noise in agentic program repair [2]. We answer the question on public data with a benchmark, a set of strong baselines, and three abstention mechanisms.

What we find. The agent does better exactly where the task is hard. It reaches 78.9% overall and 71.2% on the reassigned bugs, against 55.6% for the best classical model and 32.8% for the reporter baseline, and it has the best risk-coverage curve of any method we evaluate (AURC 0.077 against 0.184). Its abstention points the right way, but only just: a lift of 1.13, against 0.69–0.90 for every classical method (Table 7). Retrieval is not what earns the accuracy. Single-round RAG is the weakest of the three LLM conditions at 68.0%, below the 70.9% the same model reaches with no retrieval at all (Table 5); it is the tool loop, not the extra context, that helps.

Contributions.

- A public benchmark for team-level routing built from the Bugzilla API, with two test periods and a weak label for ownership ambiguity derived from reassignment history (Section 3).
- The observation that the component the reporter already chose is the strongest single signal (83.1%) and that overall accuracy therefore hides the subgroup where routing is actually hard, together with the per-category accuracy and abstention analysis that exposes it (Table 7).
- Eight classical baselines and three LLM conditions scored by one evaluator, showing that a tool-using agent is both the most accurate method on the reassigned subgroup (71.2%) and the only learned method whose abstention points the right way there, by a margin narrower than its accuracy gain (lift 1.13).
- A cost and configuration study from measured token counts, including the measurement that the model's thinking mode

buys no accuracy on this task while multiplying output tokens by five to nineteen (Table 4).

2 RELATED WORK

Routing as classification. Component and developer assignment has been studied as text classification for two decades. Sarkar et al. found at Ericsson that a classifier is accurate enough to be useful only above a high confidence threshold, and that everything below it should go to a human [13]. BugBug, in production for Firefox, applies the same idea with a 60% threshold over an XGBoost model [3]. Later work adds structure to the text: Su et al. learn from earlier misroutes with a bug-tossing knowledge graph [16], and single out confusable components by distinguishing reports that were reassigned from those that were not [15]. Transformer representations improve on TF-IDF [6], and ensembles of fine-tuned models improve further [5]. We keep this line of work as baselines (TF-IDF, embeddings, XGBoost) and add the signal these papers do not compare against: the component the reporter already chose.

Agentic and LLM triage. Instruction-tuned models with constrained decoding reach a Hit@10 of 0.753 while exact top-1 accuracy stays modest [9], and a benchmark of classical, transformer and RAG approaches finds the classical models hard to beat [10]. TriagerX reports a deployment in which components act as proxies for teams [11]. In incident triage, a single LLM assigns incidents to teams with an explanation [17], several agents negotiate an assignment [19], and one agent per team decides whether to accept it [18]; TianoForge does the same for firmware reports [14]. These systems are compared by assignment accuracy. Our claim is that aggregate accuracy is the wrong summary for this task, so we measure the same kind of system on the subgroup where ownership actually moved.

Abstention and selective prediction. Declining to answer is an old idea: Chow derived the optimal error-reject tradeoff [4], selective classification was later put on a risk-coverage footing [7] and extended to neural networks with a guaranteed risk [8]. Conformal prediction gives candidate sets with distribution-free coverage [1], which is what our prediction-set baseline uses. In software engineering, abstention has recently been used to cut the noise produced by agentic program repair [2]. We keep the risk-coverage framing and apply it to a label whose noise is not random: here the ambiguity comes from the ownership history of the bug.

Data and surveys. Our corpus is built directly from the public Bugzilla API. GitBugs aggregates Bugzilla, GitHub and Jira reports for related tasks [12], and a systematic review of 234 triage papers surveys the field and its evaluation practices [20].

3 DATA AND OWNERSHIP-AMBIGUITY LABELS

Corpus. We read the public Mozilla Bugzilla REST API. A bug enters the corpus when it is a `defect`, belongs to one of four products (Firefox, Core, Toolkit, DevTools), has resolution `FIXED`, and does not carry the `intermittent-failure` keyword. The component recorded at freeze time is the routing label, and BMO's own `team_name` field maps each component to the team that owns it.

Table 1: Ownership-ambiguity categories per split. Categories come from the component and product changes recorded in each bug's history; `bot_only` is excluded from the analysis because an automation move carries no evidence of human disagreement. Source: `results/label_categories.json` (`code/label_cleaning.py`).

Category	train	test	test2026
never_changed	8,916	4,538	5,807
inbox_triage	1,075	595	463
specific_reassign	831	434	687
bot_only	643	299	298
total	11,465	5,866	7,255

Only public data is used, and one script (`code/fetch_period.py`) builds both periods.

Two test periods. A single test period cannot separate contamination from difficulty, so we keep two. The 2022–2024 pull holds 17,331 bugs: we train on the 2022–2023 slice and test on the 2024 slice. The 2026 pull holds 7,255 bugs filed between 2026-01-01 and the 2026-10-12 freeze, all of them test data and all carrying a description. Keeping the 20 most frequent training teams leaves 5,469 test bugs for 2024 and 6,347 for 2026 (Table 5). Because the 2026 period post-dates the training cutoff of current language models it is the cleaner test set, and the 2024 period is kept for comparison.

Reassignment as the ambiguity signal. Bugzilla records every component and product change in a bug's history. We use those events as a weak label for ownership ambiguity, and sort bugs into four categories so that a routine triage step is not mistaken for genuine disagreement (Table 1): `never_changed` (no move at all), `inbox_triage` (moved out of `General` or `Untriaged`, i.e. a triage step), `specific_reassign` (moved away from a concrete component, the group this paper is about), and `bot_only` (every recorded move was made by an automation account, excluded because a bot move carries no evidence of human disagreement). Ambiguity candidates are a minority of every period: 687 of the 7,255 bugs in 2026 and 434 of the 5,866 in 2024.

How settled are the labels? A bug filed shortly before the freeze could be reassigned afterwards, which would make its label a moving target. Reassignments turn out to be almost immediate: the median delay between filing and the last recorded move is 0.2 days, and within the 2026 period 95.7% of moves happen within 30 days and 99.0% within 90 days of filing (`results/label_stability.json`, `code/label_stability.py`). We therefore also report a *settled* subset containing only the 2026 bugs filed on or before 2026-07-14, at least 90 days before the freeze.

Are the labels trustworthy? The categories above are derived from recorded history, so two things can go wrong: the rule can be applied wrongly, and the recorded history can misstate what happened. We check both, and the two checks are not equally strong. A reviewer who did not write the rule classified all 150 sampled bugs from their components, move count and human/bot flags, and agreed with the automatic category on 100.0% of them

(Cohen's $\kappa = 1.00$; `results/review_agreement.json`). We report that figure together with its weakness rather than as validation: the sample shown to the reviewer carries the same fields the rule reads, so re-deriving a category from it is close to restating the rule, and near-perfect agreement is the expected outcome rather than an independent confirmation. The check that does test the data is automated and reproducible. For the same 150 bugs we re-fetched the history from the Bugzilla API and compared it move by move with the copy the dataset stored (`code/verify_history.py`): 150 agree exactly, 0 disagree, 0 have gained a move since the freeze, and none lost or gained an initial component (`results/history_verification.json`). The stored history is therefore faithful to the source of truth. What remains a judgement call is a definition, not the data — namely whether a move out of a component named `General` in a product where that is a concrete component should count as triage rather than reassignment. We follow the more conservative reading, keeping those 68 bugs out of the positive class, and note that the alternative would enlarge it by about a tenth.

Validity threat: the mapping is current, not historical. Team membership comes from BMO's present component-to-team mapping, not the mapping in force when each bug was filed, because the API does not expose the historical mapping. A bug that changed component for organisational rather than technical reasons is therefore labelled by today's ownership. We treat this as a threat to construct validity and return to it in Section 8.

4 METHODS

4.1 Routing as selective prediction

A router that always answers is scored by accuracy. A router that may decline is scored by its risk-coverage curve: rank the test bugs by the model's own confidence, answer the most confident fraction (the *coverage*), and measure accuracy on what was answered. We report AURC, the mean risk over that curve, and accuracy at 50, 70, 80 and 90% coverage. Classical models and LLMs are scored by one evaluator, `code/evaluate.py`, which applies the same definitions to every condition, so the curves are comparable by construction.

4.2 Classical and strong baselines

Reporter initial component. The component the reporter filed under is available at filing time, so the team that owns it is a legitimate prediction, and it is the bar every trained model has to clear. When that team falls outside the label space we fall back to the majority class.

Majority class and *keyword rules* mined from the training split (the ten highest log-odds tokens per class). *TF-IDF with logistic regression*: 1–2 grams, at most 200k features, sublinear term frequency, with the regularisation strength chosen on the last quarter of the training period from $\{1, 4, 16\}$.

TF-IDF kNN and *sentence-embedding kNN*: the 20 nearest training bugs vote, weighted by cosine similarity; the embeddings are `all-MiniLM-L6-v2`. *XGBoost* on the same TF-IDF matrix (400 trees, depth 6). *Split-conformal prediction sets*, conformalised on the last quarter of the training period, give candidate sets at target coverages of 95, 90 and 80%. Tables 2 and 3 report these; all of them are produced by `code/baselines_ext.py` and `code/baseline_embed.py`.

4.3 LLM conditions

Three conditions share the same instructions and the same label block and differ only in what the model may look at. *Zero-shot*: the team list and the bug report. *Single-round RAG*: the same, plus the 10 nearest already-routed training bugs, retrieved with the same TF-IDF index. *Agent*: the model may call three tools before answering — search similar historical bugs, look up which team owns a component, and list a team's most recent training bugs — for at most six rounds, after which it is told to answer.

Every condition must reply with a single JSON object holding a ranked list of up to three teams, an explicit `abstain` flag, a confidence in the first team, and a one-sentence justification. We compare three ways of declining: the model's *verbalised confidence*, *multi-sample consistency* (five samples per bug, their agreement used as the confidence), and the agent's *explicit abstention*. All conditions run on two DeepSeek models, `deepseek-flash` and `deepseek-v4-pro` (Section 5).

5 EXPERIMENTAL SETUP

The label space is the 20 most frequent teams in the training split, estimated on 2022–2023 only, which leaves 11,021 training bugs. Test periods are 2024 and 2026 (Section 3). We report accuracy, macro-F1, top-3 accuracy, the risk-coverage numbers above, and, for the ambiguity analysis, accuracy and abstention split by ownership category, with paired McNemar tests between conditions on the bugs they share.

Cost and latency are recorded per bug. Every model request is cached on disk under a key derived from its exact payload, so an interrupted run replays cached answers instead of paying twice, and a spend guard aborts a run that exceeds its budget. Token counts and dollar costs are estimates built on a proxy tokenizer and list prices, and are reported as such. Because output tokens are the largest line in that bill and the API enables thinking unless it is told otherwise, we measured what the thinking mode buys before committing to a sweep (Table 4): it leaves accuracy unchanged, costs five to nineteen times the output tokens, and returns unparseable answers on 30 to 50% of bugs when the reasoning budget runs into the output cap. Every run reported here sends `thinking: disabled`, and the peak-hour price is avoided by waiting for the off-peak window.

6 RESULTS

Scope of the LLM runs. Every condition, classical or LLM, is scored on the same 6,347 bugs of the 2026 test period. The comparison is therefore paired by construction, and the tests in Table 8 use the whole set rather than an intersection. The reassigned subgroup is 656 bugs, small enough that its per-category numbers carry roughly ±8 points, and we report them as an estimate rather than a settled figure.

Routing accuracy. Every LLM condition beats every classical model. The agent reaches 78.9% against 62.5% for TF-IDF with logistic regression, 59.6% for XGBoost and 55.7% for the embedding *k*NN, and the advantage survives at the top of the list: 93.5%

Table 2: Classical baselines on the test2026 split. Source: `results/baselines_ext_test2026.json`.

Method	Acc	Macro-F1	Top-3	AURC	Acc@50%	Acc@80%
majority	9.4	0.9	35.2	0.945	6.7	10.0
keyword_rules	19.7	24.2	44.2	0.635	26.4	22.7
tfidf_logreg	62.5	55.9	84.4	0.184	82.5	69.9
tfidf_knn	53.8	49.8	77.2	0.266	73.9	61.1
embed_knn	55.7	49.4	81.0	0.253	73.8	63.2
xgboost	59.6	54.6	82.1	0.196	81.3	67.7

Table 3: Split-conformal prediction sets for TF-IDF+LR on the test2026 split. Calibration is the last quarter of the training period; the coverage gap is the finite-sample and distribution-shift effect. Source: `results/baselines_ext_test2026.json`.

α	Target cov.	Empirical cov.	Mean set size	Singleton rate
α=0.05	95.0	95.0	5.49	15.5
α=0.10	90.0	90.6	3.60	22.1
α=0.20	80.0	81.3	2.07	36.4

Table 4: What the thinking mode buys, on the same 100 bugs and the same seed. Turning it on multiplies output tokens several-fold and leaves accuracy where it was, and it costs parseable answers when the reasoning budget runs into the output cap. Source: `results/ab_thinking_deepseek-flash_n100.json` (`code/ab_thinking.py`).

Condition	Thinking	parse	Acc	Out tok	Lat. s
agent	disabled	100.0	67.7	293	6.2
agent	enabled	93.9	67.7	1442	14.6
rag	disabled	100.0	57.6	66	2.2
rag	enabled	69.7	55.6	1283	9.5
zeroshot	disabled	100.0	61.6	66	1.9
zeroshot	enabled	83.8	61.6	869	7.0

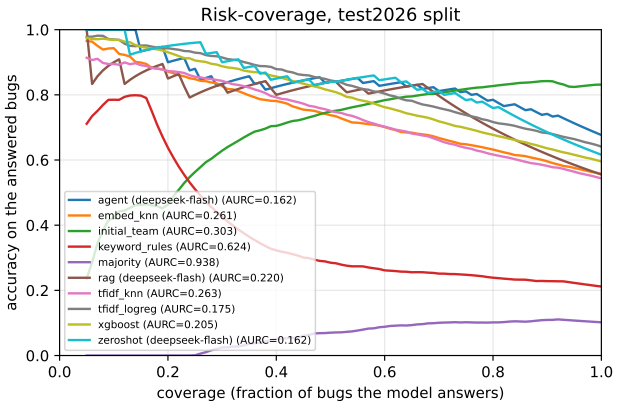


Figure 1: Risk-coverage on the 2026 test period. Source: `code/make_figures.py`.

top-3 against 84.4% (Table 5). It does not beat the reporter baseline, which stands at 83.1%. Taken alone, overall accuracy still argues for leaving the routing to whoever filed the report.

What that accuracy costs. The gain is not free, and the bill is worth stating rather than leaving in a footnote (Table 6). The agent costs $0.0047 per bug against $0.0014 for zero-shot and $0.0018 for single-round RAG, and takes 5.09 seconds per bug against 1.77 seconds. Most of both is the tool loop: the agent makes 3.57 tool calls per bug on average, which is where its 30,214 input tokens per bug come from. In absolute terms this is cheap — under half a cent per bug — so the trade is easy to accept when the reports being routed are the ambiguous tenth. It would be much harder to justify if the model were mostly recovering what the reporter already said, which is precisely what the aggregate number rewards.

The subgroup that decides it. The picture changes on the bugs whose component was moved away from a concrete one. There the reporter baseline falls to 32.8% and TF-IDF with logistic regression to 55.6%, while the agent holds 71.2% on the 656 such bugs it covered: 15.5 points above the best classical model and 38.4 points above the baseline. The distance between the two categories is also smallest for the agent — 80.0% against 71.2% — where the reporter baseline has 100.0% against 32.8%. The practical reading is that a model is worth deploying only for the fraction of reports whose ownership moved, and that is exactly the fraction overall accuracy does not isolate.

Knowing when to decline. Ordering the bugs by confidence and answering only the most confident fraction gives the agent the best risk-coverage curve of any method we evaluate (Figure 1): AURC 0.077 against 0.184 for logistic regression and 0.333 for the reporter baseline, and 86.2% correct on the 80% of bugs it chooses to answer. Its abstention rates point the right way and only just: on the reassigned subgroup it declines 22.6% of the time against 19.0% elsewhere, a lift of 1.13, where every classical method does the opposite, declining on the hard bugs at 0.69–0.90 times its own average rate. Zero-shot (0.69) and RAG (0.88) fall in between: they are more

Table 5: Every condition, scored by one evaluator so the risk-coverage curves are directly comparable. Source: `results/metrics_*.json`.

Condition	Model	Seed	*n*	Acc	Macro-F1	Top-3	AURC	Acc@80%	Abstain	$/bug
agent	deepseek-flash	0	6,347	78.9	76.0	93.5	0.077	86.2	0.3	0.00471
embed_knn	classical	–	6,347	55.7	49.4	81.0	0.253	63.2	0.0	–
initial_team	classical	–	6,347	83.1	84.5	87.4	0.333	82.7	0.0	–
keyword_rules	classical	–	6,347	19.7	24.2	44.2	0.635	22.7	0.0	–
majority	classical	–	6,347	9.4	0.9	35.2	0.945	10.0	0.0	–
rag	deepseek-flash	0	6,347	68.0	63.3	87.7	0.176	73.7	0.3	0.00177
tfidf_knn	classical	–	6,347	53.8	49.8	77.2	0.266	61.1	0.0	–
tfidf_logreg	classical	–	6,347	62.5	55.9	84.4	0.184	69.9	0.0	–
xgboost	classical	–	6,347	59.6	54.6	82.1	0.196	67.7	0.0	–
zeroshot	deepseek-flash	0	6,347	70.9	63.6	88.4	0.157	76.1	0.9	0.00143

Table 6: Cost and latency per bug on the test2026 split. Costs are the recorded API spend at off-peak list price; the classical methods run locally and cost nothing. "in" and "out" are mean prompt and completion tokens, and "calls" the mean number of tool calls per bug. A dash means the quantity does not apply. Source: `results/metrics_*.json`; the billed total, including peak-hour calls, is audited by `code/spend_report.py` into `results/spend_report.json`.

Condition	*n*	$/bug	in	out	s/bug	calls
agent (deepseek-flash)	6,347	0.0047	30214	303	5.09	3.57
zeroshot (deepseek-flash)	6,347	0.0014	9254	66	1.77	0.00
rag (deepseek-flash)	6,347	0.0018	11532	65	1.69	0.00
embed_knn	6,347	–	–	–	–	–
initial_team	6,347	–	–	–	–	–
keyword_rules	6,347	–	–	–	–	–
majority	6,347	–	–	–	–	–
tfidf_knn	6,347	–	–	–	–	–
tfidf_logreg	6,347	–	–	–	–	–
xgboost	6,347	–	–	–	–	–

Table 7: Accuracy and confidence-based abstention by ownership-ambiguity category on the full test2026 split. Every classical method is less accurate on `specific_reassign` than on `never_changed` and abstains no more often there (lift below 1), so its confidence does not track the ambiguity. The agent abstains more often on it (lift 1.21) and is 14.4 points more accurate there than the best classical model. Categories come from `code/label_cleaning.py`.

Condition	Model	Acc never	Acc specific	Gap	Abst. specific	Abst. never	Lift	Pred=init	Err=init
agent	deepseek-flash	80.0	71.2	8.8	22.6	19.0	1.13	41.6	49.7
embed_knn	classical	55.6	51.5	4.0	18.0	20.8	0.90	34.0	24.8
initial_team	classical	100.0	32.8	67.2	33.1	21.2	1.65	88.4	84.8
keyword_rules	classical	19.7	21.5	-1.8	15.7	20.4	0.79	11.6	4.3
majority	classical	8.5	14.0	-5.5	17.4	20.3	0.87	8.2	3.7
rag	deepseek-flash	68.9	61.1	7.8	17.7	19.9	0.88	39.3	37.6
tfidf_knn	classical	54.7	49.8	4.9	16.9	20.4	0.85	32.9	22.8
tfidf_logreg	classical	63.0	55.6	7.4	16.8	21.3	0.84	36.9	29.6
xgboost	classical	60.2	52.9	7.3	13.9	21.6	0.69	35.2	27.5
zeroshot	deepseek-flash	72.4	63.7	8.7	13.9	20.5	0.69	38.1	35.3

accurate than the classical models, but their confidence is, like the classical models', backwards.

6.1 Robustness

A second test period. The reversal we report is not a property of one data slice. The 2024 test period was collected from the same tracker two years earlier, and every condition in Table 9 runs locally, so the comparison costs nothing to repeat. The pattern is

Table 8: Pairwise McNemar exact tests on top-1 correctness, test2026 split. Each row uses only the bugs that both conditions answered, so n differs between rows; "A only" and "B only" count the bugs that only the first and only the second condition got right. Source: `results/significance_test2026.json`.

Comparison	n shared	A only	B only	p
agent@deepseek-flash vs rag@deepseek-flash	6,347	978	287	1.8e-88
agent@deepseek-flash vs zeroshot@deepseek-flash	6,347	852	346	1.1e-49
rag@deepseek-flash vs zeroshot@deepseek-flash	6,347	662	847	2.1e-06
agent@deepseek-flash vs embed_knn	6,347	1,765	296	4.6e-254
agent@deepseek-flash vs initial_team	6,347	767	1,039	1.7e-10
agent@deepseek-flash vs keyword_rules	6,347	3,825	72	0
agent@deepseek-flash vs majority	6,347	4,470	63	0
agent@deepseek-flash vs tfidf_knn	6,347	1,864	273	1.2e-290
agent@deepseek-flash vs tfidf_logreg	6,347	1,383	342	3.9e-148
agent@deepseek-flash vs xgboost	6,347	1,556	331	2e-189

identical in both periods: the reporter baseline is the most accurate method overall (80.3% and 83.1%), it is exactly right wherever ownership never moved (100.0% in both), and it collapses on the bugs whose component was moved away from a concrete one, falling below TF-IDF with logistic regression — 28.0% against 50.9% in 2024, and 32.8% against 55.6% in 2026. What changed between the periods is the size of the subgroup: the share of bugs whose ownership moved grew from 7.5% to 10.3%, so the part overall accuracy hides is getting larger, not smaller.

Labels that had time to settle. A bug filed shortly before the freeze could still be reassigned, which would make its label a moving target as we watch it. The settled subset keeps only bugs filed at least 90 days before the freeze and repeats the comparison (Table 10); it is unchanged.

7 ERROR ANALYSIS

Where the mistakes go. The mistakes of TF-IDF with logistic regression are not spread evenly over the 20 teams. Table 12 lists its ten most frequent confusions, and the largest by a wide margin is Frontend against Search Next Generation: 217 bugs owned by Frontend are answered with Search Next Generation, and 87 go the other way. The next groups are Layout against GFX (65 and 60) and DOM Core against Layout (64 and 47). Each of these is a pair of teams whose reports describe the same user-visible symptoms, which suggests the remaining error is a vocabulary problem rather than a coverage problem: more training data of the same kind would not separate them.

Where the agent loses. Comparing two accuracy percentages cannot say where a difference comes from, so Table 11 counts the disagreements directly, over the 6,347 bugs that both the agent and TF-IDF with logistic regression answered. The agent is right where logistic regression is wrong on 1,383 bugs and the reverse on 342. The margin is about the same in both ownership categories — 17.0 points on the bugs whose ownership never moved and 15.6 on the ones where it did — which says the agent is not merely better on the hard subgroup but better everywhere, as one would expect from a model that reads more of the report. The subgroup still matters, for a different reason: it is where the reporter baseline fails, and no aggregate accuracy reveals that. Where the agent does lose, the losses follow team size rather than area. Frontend (259 bugs), DOM Core (108) and Search Next Generation (68) are also the three teams with the most bugs in the test period, so the losses are a volume effect rather than a gap in one part of the tracker.

A strong model is pulled towards the answer that was overruled. Section 3 labels every bug by whether its component moved. Where it did not, the reporter's initial team is by definition the final team, so reproducing it can never be a mistake. Where the bug was reassigned away from a concrete component the picture reverses: the last column of Table 7 gives the share of a model's mistakes on that subgroup that land on the team the reporter first filed under, i.e. exactly the choice that was overruled. TF-IDF with logistic regression does so in 29.6% of its mistakes there, XGBoost in 27.5%, the sentence-embedding kNN in 24.8% and TF-IDF kNN in 22.8%. The ordering follows model quality exactly (53.8%, 55.7%, 59.6% and 62.5% overall accuracy in the same order, from Table 5): the better the model, the more of its remaining errors are the reporter's original choice. A weak model fails for its own reasons; a strong one converges on what the reporter said before a triager moved the bug.

This is the behaviour the LLM conditions were meant to avoid: a model that only reads the report can recover the pre-reassignment answer, because that answer is written into the report. The answer from Section 6 is that the agent does not avoid it. On the reassigned subgroup 49.7% of the agent's mistakes land on the team the reporter first filed under, which is higher than any classical model and well above the 29.6% of logistic regression. What the agent buys is not immunity to the reporter's original choice but a higher rate of getting past it: it reaches 71.2% on that subgroup against 55.6%, so it recovers from the superseded answer more often rather than never falling for it. That distinction matters for anyone deploying this: the agent does not stop being misled by the report, it just notices more often.

8 LIMITATIONS

The team mapping is current, not historical. The Bugzilla API does not expose the component-to-team mapping that was in force

Table 9: The reversal on two independent test periods. "Never" and "Moved" are the accuracy on bugs whose component was never reassigned and on bugs whose component was moved away from a concrete one. Both periods come from the same tracker two years apart and the pattern is identical: the reporter baseline wins overall, is perfect where ownership never moved, and collapses where it did, falling below TF-IDF with logistic regression. Source: `results/metrics_*.json`, `results/baselines_ext_*.json`.

Period	Condition	*n*	Acc	Never	Moved	*n* moved
2024	initial_team	5,469	80.3	100.0	28.0	411
	keyword_rules	5,469	23.6	23.8	16.5	411
	majority	5,469	9.5	9.5	9.2	411
	tfidf_knn	5,469	54.6	56.4	42.1	411
	tfidf_logreg	5,469	62.5	63.3	50.9	411
2026	initial_team	6,347	83.1	100.0	32.8	656
	keyword_rules	6,347	19.7	19.7	21.5	656
	majority	6,347	9.4	8.5	14.0	656
	tfidf_knn	6,347	53.8	54.7	49.8	656
	tfidf_logreg	6,347	62.5	63.0	55.6	656

Table 10: Accuracy and confidence-based abstention by ownership-ambiguity category on the settled subset, i.e. bugs filed at least 90 days before the freeze. Every classical method is less accurate on `specific_reassign` than on `never_changed` and abstains no more often there (lift below 1), so its confidence does not track the ambiguity. The agent abstains more often on it (lift 1.21) and is 14.4 points more accurate there than the best classical model. Categories come from `code/label_cleaning.py`.

Condition	Model	Acc never	Acc specific	Gap	Abst. specific	Abst. never	Lift	Pred=init	Err=init
keyword_rules	classical	21.2	22.6	-1.4	17.9	20.1	0.89	12.4	4.6
majority	classical	9.3	14.9	-5.6	18.3	20.0	0.91	9.2	4.2
tfidf_knn	classical	55.1	51.6	3.5	17.5	20.5	0.88	33.0	21.0
tfidf_logreg	classical	64.4	58.8	5.6	18.3	21.2	0.91	37.7	28.3

Table 11: Where the agent and TF-IDF with logistic regression disagree, over the 6,347 bugs both answered. "Net" is the agent's advantage in accuracy points on that row. The advantage is roughly twice as large per bug where ownership moved, and the teams the agent loses to are the largest ones, so the losses are a volume effect rather than a gap in one area. Source: `llm_agent_deepseek-flash_test2026_disabled_n6347_seed0.jsonl`, `pred_tfidf_logreg_test2026.jsonl`.

Subset	*n*	Agent only	LR only	Net
All shared bugs	6,347	1,383	342	+16.4
Ownership never moved	4,958	1,094	253	+17.0
Ownership moved away	656	138	36	+15.6

when a bug was filed, so we label every bug with today's ownership. A bug that moved component for organisational reasons is therefore labelled by current ownership, and a team renamed or split since 2022 is invisible to us. This is the main threat to construct validity in this paper, and it is the reason we report two test periods rather than one.

Reassignment is a proxy for ambiguity, not a measurement of it. We infer ownership ambiguity from the fact that a bug's component changed. Some of those moves are routine: a triager can move a report to a more specific component without anyone disagreeing about which team owns it. We separate the inbox cases and the automation cases out of the candidate set for exactly this reason, but the residual group is still a proxy, so we treat the per-category numbers as descriptive rather than as a gold-standard evaluation.

The reporter baseline is not a human baseline. We do not model who filed each report. Many Bugzilla reports come from developers or from automation rather than from a user meeting the product for the first time, so trusting the initial component is an upper bound on what "trust the filer" can achieve, not a model of a human triager.

One project, one split, one model family. All numbers come from Mozilla Bugzilla, with a single train/test split and a 20-team label space chosen by frequency. The settled subset is a stability check on the labels, not cross-validation across time. The LLM conditions use one provider, so model quality and prompt design cannot be separated from each other.

Costs and token counts are estimates. Dollar figures apply list prices to token counts produced by a proxy tokenizer rather than

Table 12: The ten most frequent confusions of `tfidf_logreg` on the test2026 split: the team that owns the bug, and the team the model named first. Source: `results/metrics_tfidf_logreg_test2026.json` (`code/evaluate.py`).

True team	Predicted first	n
Frontend	Search Next Generation	217
Search Next Generation	Frontend	87
Layout	GFX	65
DOM Core	Layout	64
GFX	Layout	60
Frontend	OS Integration	59
Media	GFX	58
Frontend	Layout	47
Layout	DOM Core	47
OS Integration	GFX	41

by the provider's own tokenizer, and prompt caching, off-peak pricing and retries all move the real bill. Latency includes network time. We report them to bound the cost of the method, not as a measurement of it.

9 REPRODUCIBILITY

The whole pipeline runs from one command, `code/run_all.py`, which calls the other scripts in order and reports what each one wrote; the paid steps run only when it is given `--with-llm` and an API key is present. Data is pulled from the public Bugzilla REST API by `code/fetch_period.py`, so the corpus can be rebuilt from scratch rather than taken on trust. `code/requirements.txt` pins every dependency.

One discrepancy is worth recording. The TF-IDF with logistic regression baseline scores 62.443% on the 2024 test period on the machine where the first results were produced and 62.534% on the machine used here, on the same dataset; we attribute the difference to library versions. The original outputs are kept in `results/` next to the new ones instead of being overwritten, and the pinned versions in `code/requirements.txt` are what the reported numbers were produced with.

Every model response is cached on disk under a key derived from the exact request payload, so a run can be interrupted and resumed without paying for the same request twice, and a spend guard aborts a run that exceeds its budget. Given that cache, prompt construction, response parsing and scoring are deterministic.

10 CONCLUSION

A router is usually judged by how often it names the right team. On this benchmark that measure picks the wrong winner: predicting the team that owns the component the reporter already chose scores 83.1%, better than any model, but it collapses to 32.8% on the reports whose ownership actually moved, which is the only subgroup where routing is still undecided. A tool-using LLM agent takes that subgroup to 71.2%, 15.5 points above the best classical model, and it is the one learned method whose confidence is higher there than elsewhere, so it can be spent selectively where a human reviewer is worth the most. Aggregate accuracy hides all of this, and we would suggest that routing results be reported per ownership category from now on.

Two threats to validity remain. The component-to-team mapping we use is the current one, not the historical one, so a bug whose component changed for organisational rather than technical reasons is labelled by today's ownership; an organisational mapping would be a better label. And the agent is priced in tokens here; whether it pays for itself depends on how much triage time it saves, which a token bill does not measure.

ACKNOWLEDGMENT

The author used an AI coding assistant — an agentic tool loop driving DeepSeek and Gemini language models — while preparing this paper, and remains solely responsible for all content, claims and references. The assistant drafted and revised the prose of every section of the manuscript, and wrote the analysis scripts in the replication package; the tables and figures are produced by those scripts from recorded results rather than composed by the assistant, and every reference was checked against the published record. The language models evaluated in Section 5 are the object of study rather than writing assistance, and are described there.

REFERENCES

[1] Anastasios N. Angelopoulos and Stephen Bates. 2021. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. https://doi.org/10.48550/arXiv.2107.07511 arXiv:2107.07511 [cs.LG] verified 2026-10-12 (arXiv abstract page; v1 submitted 2021-07-15, v6 revised 2022-12-07). Year is the first-submission year.

[2] José Cambronero, Michele Tufano, Sherry Shi, Renyao Wei, Grant Uy, Runxiang Cheng, Chin-Jung Liu, Shiying Pan, Satish Chandra, and Pat Rondon. 2026. Abstain and Validate: A Dual-LLM Policy for Reducing Noise in Agentic Program Repair. In *2026 IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. https://doi.org/10.1145/3786583.3786858 arXiv:2510.03217 verified 2026-10-12 (arXiv abstract page states acceptance to ICSE-SEIP '26 and gives the related DOI).

[3] Marco Castelluccio and Sylvestre Ledru. 2019. Teaching machines to triage Firefox bugs. Mozilla Hacks (Mozilla developer blog). https://hacks.mozilla.org/2019/04/teaching-machines-to-triage-firefox-bugs/ verified 2026-10-12 (fetched the post; posted 2019-04-09). Not peer reviewed: this is the production description of BugBug, including its 60% confidence threshold and XGBoost model.

[4] C. K. Chow. 1970. On Optimum Recognition Error and Reject Tradeoff. *IEEE Transactions on Information Theory* (1970). https://doi.org/10.1109/TIT.1970.1054406 verified 2026-10-12 (Semantic Scholar API; DBLP key journals/tit/Chow70). Volume and page numbers were NOT confirmed and are therefore omitted rather than guessed.

[5] Atish Kumar Dipongkor. 2024. An Ensemble Method for Bug Triaging Using Large Language Models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. https://doi.org/10.1145/3639478.3641228 verified 2026-10-12 (Semantic Scholar API; DBLP key conf/icse/Dipongkor24a).

[6] Atish Kumar Dipongkor and Kevin Moran. 2023. A Comparative Study of Transformer-based Neural Text Representation Techniques on Bug Triaging. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. https://doi.org/10.48550/arXiv.2310.06913 arXiv:2310.06913 [cs.SE] verified 2026-10-12 (arXiv abstract page states "to appear in the Proceedings of 38th IEEE/ACM ASE'23").

[7] Ran El-Yaniv and Yair Wiener. 2010. On the Foundations of Noise-free Selective Classification. *Journal of Machine Learning Research* 11, 53 (2010), 1605–1641. verified 2026-10-12 (OpenAlex: JMLR volume 11, issue 53, pages 1605–1641).

[8] Yonatan Geifman and Ran El-Yaniv. 2017. Selective Classification for Deep Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1705.08500 verified 2026-10-12 (OpenAlex locates the record in the NeurIPS proceedings at papers.nips.cc with a Technion affiliation; the arXiv abstract page confirms title, authors and year).

[9] Kiana Kiashemshaki, Arsham Khosravani, Alireza Hosseinpour, and Arshia Akhavan. 2025. Automated Bug Triaging using Instruction-Tuned Large Language Models. https://doi.org/10.48550/arXiv.2508.21156

arXiv:2508.21156 [cs.SE] verified 2026-10-12 (arXiv abstract page). Preprint: no peer-reviewed venue stated on the page.

[10] Márk Lajkó, Balázs Nagy, and László Vidács. 2026. Benchmarking Classical, Transformer-Based, and RAG Approaches with LLMs for Automated Bug Triage. In *Proceedings of the 21st International Conference on Software Technologies (ICSOFT)*. verified 2026-10-12 for title, authors and year (read from the paper PDF). VENUE NOT INDEPENDENTLY CONFIRMED: ICSOFT 2026 comes from the step-1 report, the PDF itself carries no venue line. Re-check before camera-ready.

[11] Md Afif Al Mamun, Gias Uddin, Lan Xia, and Longyu Zhang. 2026. TriagerX: Dual Transformers for Bug Triaging Tasks with Content and Interaction Based Rankings. *IEEE Transactions on Software Engineering* (2026). https://doi.org/10.1109/TSE.2026.3685573 arXiv:2508.16860 verified 2026-10-12 (arXiv abstract page states acceptance to IEEE TSE and gives the related DOI).

[12] Avinash Patil, Siru Tao, and Aryan Jadon. 2025. GitBugs: Bug Reports for Duplicate Detection, Retrieval Augmented Generation, Triage, and More. https://doi.org/10.48550/arXiv.2504.09651 arXiv:2504.09651 [cs.SE] verified 2026-10-12 (arXiv abstract page, v4 dated 2026-04-28).

[13] Aindrila Sarkar, Peter C. Rigby, and Béla Bartalos. 2019. Improving Bug Triaging with High Confidence Predictions at Ericsson. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. https://doi.org/10.1109/ICSME.2019.00018 verified 2026-10-12 (Semantic Scholar API, DOI 10.1109/ICSME.2019.00018; DBLP key conf/icsm/SarkarRB19).

[14] Nazanin Siavash, Terrance E. Boult, and Armin Moin. 2026. TianoForge: An Automated Bug Triage Approach for the TianoCore UEFI Firmware Development Community. https://doi.org/10.48550/arXiv.2608.23259 arXiv:2608.23259 [cs.SE] verified 2026-10-12 (arXiv abstract page). The page states the work was accepted for presentation at the FTA 2026 workshop but is deliberately NOT in the proceedings.

[15] Yanqi Su, Zheming Han, Zhipeng Gao, Zhenchang Xing, Qinghua Lu, and Xiwei Xu. 2023. Still Confusing for Bug-Component Triaging? Deep Feature Learning and Ensemble Setting to Rescue. In *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*. https://doi.org/10.1109/ICPC58990.2023.00046 verified 2026-10-12 (OpenAlex).

[16] Yanqi Su, Zhenchang Xing, Xin Peng, Xin Xia, Chong Wang, Xiwei Xu, and Liming Zhu. 2021. Reducing Bug Triaging Confusion by Learning from Mistakes with a Bug Tossing Knowledge Graph. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. https://doi.org/10.1109/ASE51524.2021.9678574 verified 2026-10-12 (OpenAlex; title and authors also read from the paper PDF). The step-1 report called this "LR-BKG"; that is a shorthand, not the title.

[17] Zexin Wang, Jianhui Li, Minghua Ma, Ze Li, Yu Kang, Chaoyun Zhang, Chetan Bansal, Murali Chintalapati, Saravan Rajmohan, Qingwei Lin, Dongmei Zhang, Changhua Pei, and Gaogang Xie. 2024. Large Language Models Can Provide Accurate and Interpretable Incident Triage. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. verified 2026-10-12 (title, full author list and affiliations read from the paper PDF hosted by Microsoft Research at /en-us/research/wp-content/uploads/2024/08/ISSRE24_-LLM4triage.pdf, which encodes ISSRE 2024 in its name). The step-1 report called this "COMET"; that is a shorthand, not the title.

[18] Yuhan Yao, Yuxuan Jiang, Ming-Hua Ma, Madhura Vaidya, Jieren Deng, Yigong Hu, Chetan Bansal, Ze Li, and Murali Chintalapati. 2026. An Agentic Framework for Triaging Incidents in Production Cloud Infrastructure. In *Companion Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. https://doi.org/10.1145/3803437.3805228 verified 2026-10-12 (Semantic Scholar API, DOI 10.1145/3803437.3805228; DBLP key conf/sigsoft/YaoJMVDHBLC26). The step-1 report called this "Comfey"; that name does not appear in the title.

[19] Zhaoyang Yu, Minghua Ma, Xiaoyu Feng, Ruomeng Ding, Chaoyun Zhang, Ze Li, Murali Chintalapati, Xuchao Zhang, Rujia Wang, Chetan Bansal, Saravan Rajmohan, Qingwei Lin, Shenglin Zhang, Changhua Pei, and Dan Pei. 2025. Triangle: Empowering Incident Triage with Multi-LLM-Agents. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. verified 2026-10-12 (title and full author list read from the paper PDF hosted by Microsoft Research; the file is named TRIANGLE_FSE25.pdf, which is where the FSE 2025 attribution comes from). OpenAlex did not return this record, so the venue is not independently confirmed.

[20] Yongxin Zhao, Shenglin Zhang, Yujia Wu, Yuxin Sun, Yongqian Sun, Dan Pei, Chetan Bansal, and Minghua Ma. 2025. Triage in Software Engineering: A Systematic Review of Research and Practice. https://doi.org/10.48550/arXiv.2511.08607 arXiv:2511.08607 [cs.SE] verified 2026-10-12 (arXiv abstract page; reviews 234 papers). Preprint: no peer-reviewed venue stated.